

Strassen Multisystolic Array Hardware Architectures

Trevor E. Pogue  and Nicola Nicolici , *Senior Member, IEEE*

Abstract—While Strassen’s matrix multiplication algorithm reduces the complexity of naive matrix multiplication, general-purpose hardware is not suitable for achieving the algorithm’s promised theoretical speedups. This leaves the question of if it could be better exploited in custom hardware architectures designed specifically for executing the algorithm. However, there is limited prior work on this and it is not immediately clear how to derive such architectures or if they can ultimately lead to real improvements. We bridge this gap, presenting and evaluating new systolic array architectures that efficiently translate the theoretical complexity reductions of Strassen’s algorithm directly into hardware resource savings. Furthermore, the architectures are multisystolic array designs that can multiply smaller matrices with higher utilization than single-systolic array designs. The proposed designs implemented on FPGA reduce DSP requirements by a factor of 1.14^r for r implemented Strassen recursion levels, and otherwise require overall similar soft logic resources when instantiated to support matrix sizes down to 32×32 and 24×24 at 1-2 levels of Strassen recursion, respectively. We evaluate the proposed designs both in isolation and in an end-to-end machine learning accelerator compared to baseline designs and prior works, achieving state-of-the-art performance.

Index Terms—Hardware architecture, machine learning, matrix multiplication, performance, Strassen, systolic arrays.

I. INTRODUCTION

DUE to the rising demand for optimized hardware acceleration of general matrix multiplication (GEMM), the field of hardware design continues to see innovation for ways of better exploiting the inherent parallelism to speed up computation. However, at a certain point, after technology scaling slows to a halt and the system-level optimizations and known parallelism are exhausted, an accelerator wall exists which limits further progress on the implementation side [1]. A less-explored path for advancement beyond this wall is through reducing the computation at the algebraic level, by computing the same output from a re-arranged compute pattern requiring fewer or cheaper operations to be executed in hardware.

One of the area-dominant computational resources in GEMM and deep learning accelerators can commonly be the multiply-accumulate (MAC) units [2], [3], [4], and an accelerator’s throughput can be directly limited by how many MAC units can be afforded in its hardware budget. As a result, surpassing this performance per MAC limit has been focused on recently with minimal filtering algorithms applied to convolutional neural networks [2], [5] and with application of fast inner-product algorithms for speeding up deep learning and GEMM workloads [6].

The Strassen matrix multiplication algorithm [7] can also theoretically be used to reduce the complexity of naive matrix multiplication. However, its execution on general-purpose central processing units (CPU)s and graphics processing units (GPU)s has been shown to be not suitable for achieving the algorithm’s promised theoretical speedups [8], [9], [10], [11], [12], [13], [14], [15]. Strassen’s algorithm even increases execution time on CPUs/GPUs unless the matrix widths/heights are in the range of at least 1024 elements or larger. This limits the benefits of using the algorithm on these devices for modern workloads that do not decompose to such large matrix multiplications. Strassen’s algorithm contains hidden overheads such as extra data accesses required for reading/computing/storing additional intermediate matrices before/after the matrix multiplication steps. These extra steps all add to the overall execution time beyond what is expected from a theoretical analysis based on the number of arithmetic operations performed alone.

This then leaves questions surrounding if the promised theoretical complexity reductions can be more efficiently achieved in custom hardware architectures designed specifically for executing Strassen’s algorithm. However, prior work on this topic is limited and it is not immediately clear how to design such architectures or if they can truly lead to real improvements. In this work, we bridge this gap by presenting and evaluating new systolic array hardware architectures for efficiently exploiting Strassen’s algorithm. The proposed architectures achieve a more efficient implementation of Strassen’s algorithm compared to what is possible through execution on CPUs and GPUs by pipelining and performing the extra data movement and addition steps at all levels of recursion in parallel with the matrix multiplications. The Strassen architectures are functionally equivalent to conventional multisystolic array designs while allowing the theoretical complexity reductions of Strassen’s algorithm to be translated directly into hardware resource savings, even for multiplication of small matrices. Furthermore, the architectures are multisystolic array designs, which is a type of design that can multiply smaller matrices with higher utilization than a single-systolic array design.

Compared to a conventional multisystolic array design, the proposed architecture implemented on FPGA uses $1.3 \times$ fewer DSP units and a similar amount of soft logic resources when instantiated for multiplying matrix sizes down to 24×24 at 2 levels of Strassen recursion. We demonstrate how the proposed systolic array architectures are able to increase conventional multiplications/multiplier/clock cycle limits while also allowing the design to scale up in size without increasing the minimum supported matrix sizes.

T. E. Pogue and N. Nicolici are with the Department of Electrical and Computer Engineering, McMaster University, Hamilton, ON, L8S 4L8, Canada
Email: pogue@mcmaster.ca; nicolici@mcmaster.ca

II. BACKGROUND AND RELATED WORK

A. Conventional Matrix Multiplication

A conventional matrix multiplication algorithm computes $\mathbf{C} = \mathbf{AB}$ for $\mathbf{A}$ of size $M \times K$ and $\mathbf{B}$ of size $K \times N$, where each element $c_{i,j}$ of $\mathbf{C}$ is calculated as follows:

$$c_{i,j} = \sum_{k=1}^K a_{i,k} b_{k,j}. \quad (1)$$

Alternatively, $\mathbf{C}$ can also be computed by dividing $\mathbf{A}$ and $\mathbf{B}$ into 4 matrix blocks, where $\mathbf{C}$ is then computed by carrying out 8 matrix block multiplications and 4 matrix block additions between the $\mathbf{A}$ and $\mathbf{B}$ blocks as follows:

$$\begin{bmatrix} \mathbf{C}_{11} & \mathbf{C}_{12} \\ \mathbf{C}_{21} & \mathbf{C}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{bmatrix}. \quad (2)$$

This process can then be carried out recursively again for each matrix block product by splitting the matrix blocks again into smaller blocks and repeating the same process.

B. Strassen Matrix Multiplication

Strassen's fast matrix multiplication algorithm [7] provides a way to carry out (2) instead using 7 matrix block multiplications and 18 matrix block additions as follows:

$$\begin{aligned} \mathbf{T}_1 &= \mathbf{A}_{11} + \mathbf{A}_{22} & \mathbf{S}_1 &= \mathbf{B}_{11} + \mathbf{B}_{22} \\ \mathbf{T}_2 &= \mathbf{A}_{21} + \mathbf{A}_{22} & \mathbf{S}_2 &= \mathbf{B}_{11} \\ \mathbf{T}_3 &= \mathbf{A}_{11} & \mathbf{S}_3 &= \mathbf{B}_{12} - \mathbf{B}_{22} \\ \mathbf{T}_4 &= \mathbf{A}_{22} & \mathbf{S}_4 &= \mathbf{B}_{21} - \mathbf{B}_{11} \\ \mathbf{T}_5 &= \mathbf{A}_{11} + \mathbf{A}_{12} & \mathbf{S}_5 &= \mathbf{B}_{22} \\ \mathbf{T}_6 &= \mathbf{A}_{21} - \mathbf{A}_{11} & \mathbf{S}_6 &= \mathbf{B}_{11} + \mathbf{B}_{12} \\ \mathbf{T}_7 &= \mathbf{A}_{12} - \mathbf{A}_{22} & \mathbf{S}_7 &= \mathbf{B}_{21} + \mathbf{B}_{22} \end{aligned} \quad (3)$$

$$\begin{aligned} \mathbf{Q}_1 &= \mathbf{T}_1 \cdot \mathbf{S}_1 \\ \mathbf{Q}_2 &= \mathbf{T}_2 \cdot \mathbf{S}_2 \\ \mathbf{Q}_3 &= \mathbf{T}_3 \cdot \mathbf{S}_3 \\ \mathbf{Q}_4 &= \mathbf{T}_4 \cdot \mathbf{S}_4 \\ \mathbf{Q}_5 &= \mathbf{T}_5 \cdot \mathbf{S}_5 \\ \mathbf{Q}_6 &= \mathbf{T}_6 \cdot \mathbf{S}_6 \\ \mathbf{Q}_7 &= \mathbf{T}_7 \cdot \mathbf{S}_7 \end{aligned} \quad \begin{aligned} \mathbf{C}_{11} &= \mathbf{Q}_1 + \mathbf{Q}_4 - \mathbf{Q}_5 + \mathbf{Q}_7 \\ \mathbf{C}_{12} &= \mathbf{Q}_3 + \mathbf{Q}_5 \\ \mathbf{C}_{21} &= \mathbf{Q}_2 + \mathbf{Q}_4 \\ \mathbf{C}_{22} &= \mathbf{Q}_1 - \mathbf{Q}_2 + \mathbf{Q}_3 + \mathbf{Q}_6 \end{aligned} \quad (4)$$

Similarly to (2), this algorithm can also be repeated recursively for each matrix block multiplication, leading to an asymptotic complexity reduction compared to conventional matrix multiplication algorithms such as (1) and (2).

1) *Winograd Form*: The Winograd form of the Strassen algorithm [16] has the same asymptotic complexity but requires 15 matrix block additions at each level of recursion rather than 18. However, for fixed-point data types, this form increases the multiplier input datapath bitwidth by up to 2 bits for each recursion level implemented rather than 1 bit, which reduces the implementation benefits. Due to this, we focus on the original form of the Strassen algorithm from (3)-(4) in our work instead.

C. Prior Work on Multisystolic Array Systems

Systolic arrays, which we also refer to as matrix multiplication units (MXU)s for convenience, are an effective choice for use in GEMM accelerators as they significantly reduce the required memory traffic and can reach high clock frequencies due to their short and regular interconnects. Systolic array architectures have been used in state-of-the-art GEMM and deep learning accelerators such as the Tensor Processing Unit (TPU) [3], [4], [17], among others [6], [18]. However, a systolic array can only be fully utilized when the input matrix sizes at minimum match the dimensions of the systolic array or are larger, and real workloads have limits to the matrix sizes being multiplied.

There is then a limit to how fast the workload can be accelerated on a single-systolic array design. This is because, even if more compute resources are instantiated to scale up the size of the systolic array, the systolic array will begin to be underutilized after its size surpasses the workload's matrix sizes, and the workload will not be able to execute any faster. This is particularly true in modern workloads such as deep learning acceleration, where the matrix sizes that the workloads break down to can be smaller than the maximum systolic array size that could be instantiated in an accelerator [4], [17], [19], [20].

To combat this, multiple smaller systolic arrays can be used in parallel, which allows for the total compute power in the systolic array system to increase while the minimum supported matrix sizes remain the same. Prior works [19], [20] achieve this by implementing variations of (2) by dividing larger matrices into smaller matrix blocks, executing the smaller matrix block multiplications on multiple smaller systolic arrays. The block products are then later summed up to form the final larger matrix multiplication product. In this work, we show how to efficiently implement (3)-(4) in hardware to achieve this same goal with less hardware resources.

D. Prior Work on Executing Strassen on CPUs and GPUs

Strassen's algorithm has been well explored in prior work for execution on general-purpose CPUs and GPUs [8], [9], [10], [11], [12], [13], [14], [15]. However, its execution on CPUs and GPUs in these prior works is unable to efficiently achieve the algorithm's promised theoretical speedups unless the widths/heights of the matrices being multiplied are in the range of at least 1024 elements or even much larger.

This non-optimal execution of Strassen's algorithm in CPUs and GPUs stems from irregularities introduced in the algorithm such as extra data accesses required for reading/computing/storing additional intermediate matrices before/after the matrix multiplication steps. These irregularities all add to the overall execution time beyond what would be expected purely from a theoretical analysis of only the number of required arithmetic operations [10], [14].

1) *Theoretical Complexity Reductions of Strassen's Algorithm*: In this subsection, we establish what the expected theoretical complexity reductions of Strassen's algorithm are based on number of operations, and how the achieved speedups

in prior works on CPU/GPU Strassen implementations fall short of achieving these theoretical complexity reductions.

Letting $M = N = K = n$, the complexity of Strassen's algorithm in number of arithmetic operations is $\mathcal{O}(n^{2.8074})$ [7]. Conventional matrix multiplication (1) requires n^3 multiplications and $n^2(n-1)$ additions for the following number of total operations:

$$n^3 + n^2(n-1). \quad (5)$$

In contrast, Strassen's algorithm (3) for 1 recursion level requires $7n^3/8$ multiplications and $7n^2(n/2-1)/4 + 18n^3/8$ additions for the following number of total operations:

$$7n^3/8 + 7n^2(n/2-1)/4 + 18n^3/8. \quad (6)$$

The Winograd form of Strassen's algorithm [16] for 1 recursion level requires $7n^3/8$ multiplications and $7n^2(n/2-1)/4 + 15n^3/8$ additions for the following number of total operations:

$$7n^3/8 + 7n^2(n/2-1)/4 + 15n^3/8. \quad (7)$$

By comparing (5) to (6) and (7) for different values of n we can then see that Strassen's algorithm requires fewer operations than conventional matrix multiplication for matrix sizes of $n \geq 16$, and $n \geq 13$ for the Winograd form of Strassen's algorithm.

However, Strassen's algorithm on CPUs and GPUs in prior works only starts providing some speedups over traditional matrix multiplication for matrix sizes n of at least 20000 [8], 16384 [9], 896 [10], 5000 [11], 1536 [12], 1006 [13], and 1000 [14] [15]. This limits the applicability of Strassen's algorithm on CPUs and GPUs for modern workloads such as deep learning that do not always decompose to such large matrix multiplications.

As derived above, prior works on CPU/GPU implementations require matrix sizes of at least 896-16384 before having benefits rather than the much lower theoretical threshold of 13 or 16. In contrast, the custom Strassen hardware architectures presented in this work translate the benefits of Strassen's algorithm into hardware resource savings rather than reductions in execution time. The proposed designs more closely achieve the theoretical complexity reductions of Strassen's algorithm compared to prior works on CPU/GPU implementations. This is demonstrated in our results through the fact that the proposed architectures present area savings while achieving the same throughput/clock cycle as traditional designs even when instantiated for multiplying matrices down to size 24×24 . Additionally, for r Strassen recursion levels implemented, the proposed designs achieve $(8/7)^r$ times reduction in multipliers as expected from (3)-(4) compared to conventional designs without significant increase in other hardware components or any increase in throughput/clock cycle.

E. Prior Work on Custom Strassen Hardware Architectures

While software implementations of Strassen's algorithm on CPUs and GPUs have been well explored in prior work, custom hardware designs for efficiently exploiting the algorithm in hardware remain under-explored. A systolic array

design concept for implementing Strassen's algorithm for one level of recursion on 2×2 matrices has been proposed in the work by Elfimova et al. [21] without evaluation of an implementation. Another hardware design for implementing Strassen's algorithm for one level of recursion on 2×2 matrices has also been proposed in the work by León-Vega et al. [22], where the Strassen architecture reduced FPGA DSP usage by up to 12.5% at the expense of 25-40% increase in LUT resources to implement the additional adders.

Unlike the only two prior works on custom hardware designs for executing the Strassen algorithm, we propose architectures in this work that allow for Strassen's algorithm to be implemented on matrices larger than 2×2 . This is essential for minimizing the complexity penalty of the additional adders. Additionally, the architectures are capable of implementing multiple levels of Strassen recursion to achieve greater hardware resource savings. Furthermore, the proposed architectures allow proven traditional systolic arrays to be still used at the core. Alternatively, they can allow Strassen's algorithm to be used in combination with other hardware designs that can efficiently perform further algebraic optimizations on matrices after the Strassen portion is carried out, such as techniques from our prior work [6]. Finally, the proposed Strassen architectures are multisystolic array designs, meaning they can multiply smaller matrices with higher utilization than single-systolic array designs with the same computational strength.

F. Notation

The following notation is used throughout the remainder of this work for describing different systolic array architectures or their workloads:

- r : The number of recursion levels in (2) or (3)-(4) that are implemented in a hardware architecture.
- MM: A traditional single-systolic array implementing conventional matrix multiplication (1) in hardware.
- MM_r : A traditional multisystolic array implementing conventional blocked matrix multiplication (2) in hardware for r levels of recursion.
- SMM_r : The proposed Strassen multisystolic array implementing (3)-(4) in hardware for r levels of recursion.
- MXU: In this work, systolic arrays may also be referred to as matrix multiplication units (MXU)s for convenience.
- $(S)MM_{(r)} X \times Y$: An MM, MM_r , or SMM_r architecture may also be referred to with two numbers $X \times Y$ specified beside it. Here, X and Y represent the width and height, respectively, in number of MAC units of each MM systolic array instantiated at the lowest level of recursion in the architecture. For example, an MM 64×64 MXU (meaning $X = Y = 64$) would contain 64^2 MAC units, an MM_1 32×32 MXU (meaning $r = 1$ and $X = Y = 32$) would contain $8^1 \times 32^2$ MAC units, and an SMM_2 8×8 MXU (meaning $r = 2$ and $X = Y = 8$) would contain $7^2 \times 8^2$ multipliers.
- n : The width/height of the matrices that are being fed as inputs to a systolic array to be multiplied.

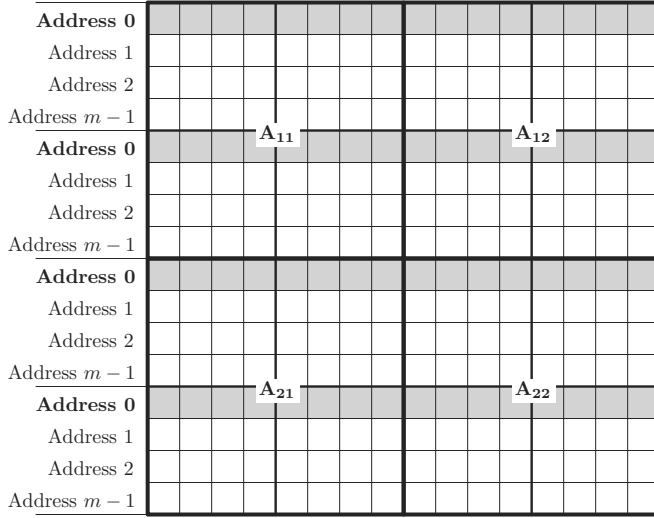


Fig. 1. Example data layout for the $\mathbf{A}$ matrix in memory for an architecture implementing Strassen matrix multiplication for 2 levels of recursion (SMM_2). Each address i contains every m^{th} row of $\mathbf{A}$ concatenated together starting at row i (notated as $\mathbf{A}_{i:m,:}$). To help illustrate this, the gray coloured rows are all elements of $\mathbf{A}$ belonging to address 0, which forms $\mathbf{A}_{0:m,:}$ containing row 0 of every $\mathbf{A}$ sub-block from the lowest level of recursion in (3). The organization for the $\mathbf{B}$ matrices in memory are the same, except that the order of the elements is transposed compared to the $\mathbf{A}$ matrix layout shown here.

III. STRASSEN ARCHITECTURE

The proposed architectures achieve a more efficient implementation of Strassen's algorithm than what is possible through execution on CPUs and GPUs by pipelining and performing the extra additions and data movement steps at all levels of recursion in parallel with the matrix multiplications. The architectures are functionally equivalent to conventional multisystolic array designs while allowing the theoretical complexity reductions of Strassen's algorithm to be translated directly into hardware resource savings.

A. Memory Layout and Access Algorithm

In order to perform the extra Strassen data movement and addition steps at all levels of recursion in parallel with the matrix multiplications, the architecture reads one row/column at a time of the $\mathbf{A}$ and $\mathbf{B}$ input matrix sub-blocks from the lowest level of recursion in (3) simultaneously. This generates and provides all $\mathbf{T}$ and $\mathbf{S}$ sub-blocks one row/column at a time for performing all the matrix multiplications in (4) at the lowest level of recursion in parallel. The $\mathbf{T}$ and $\mathbf{S}$ sub-blocks are all immediately generated from the $\mathbf{A}$ and $\mathbf{B}$ input sub-blocks and consumed in parallel like this to eliminate any additional execution time or hardware resources needed for storing/re-accessing them for later use.

To achieve this, each $\mathbf{A}$ and $\mathbf{B}$ matrix fed into the MXU is divided into 4^r equal sub-blocks of size $m \times k$ for $\mathbf{A}$ and of size $k \times n$ for $\mathbf{B}$, where each row/column i/j of each $\mathbf{A/B}$ sub-block is stored in the accelerator's $\mathbf{A}$ and $\mathbf{B}$ memories at location i/j plus an offset. An example of this memory layout for implementing 2 levels of Strassen recursion is shown in Fig. 1. This means that each $\mathbf{A}$ memory location i is a vector containing every m^{th} row of $\mathbf{A}$ starting at row i concatenated

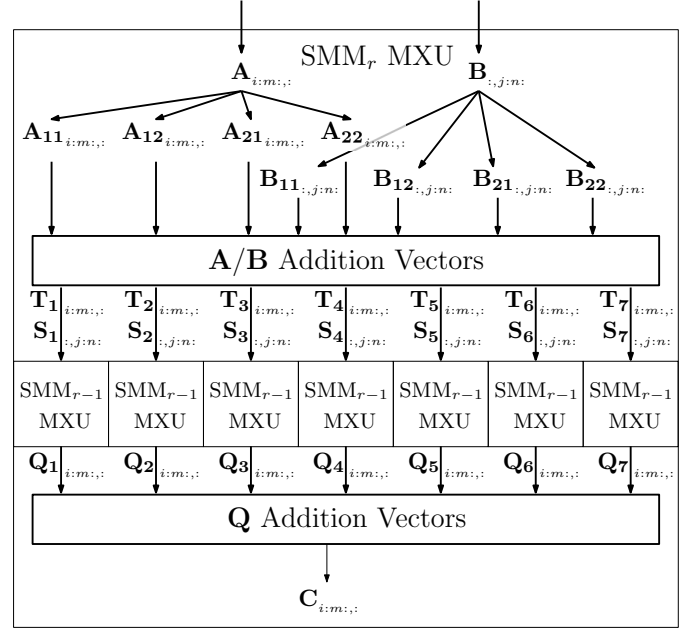


Fig. 2. Top-level diagram of the proposed SMM_r multisystolic array architecture for implementing Strassen matrix multiplication (3)-(4) for r levels of recursion in hardware.

together (notated as $\mathbf{A}_{i:m,:}$), and each $\mathbf{B}$ memory location j is a vector containing every n^{th} column of $\mathbf{B}$ starting at column j concatenated together (notated as $\mathbf{B}_{:,j:n}$). This allows one row or column of all 4^r $\mathbf{A/B}$ sub-blocks from the lowest level of recursion in (3) to all be read at once from a single memory location and fed into the MXU each clock cycle. $\mathbf{A}_{i:m,:}$ and $\mathbf{B}_{:,j:n}$ rows/columns are then read consecutively when feeding the $\mathbf{A}$ and $\mathbf{B}$ blocks into the MXU.

As shown in (3), the input $\mathbf{A}$ and $\mathbf{B}$ matrices at each level of recursion are divided into four block quadrants labelled $\mathbf{A}_{ij}$ and $\mathbf{B}_{ji}$ of size $M \times K$ for $\mathbf{A}_{ij}$ quadrants and of size $K \times N$ for $\mathbf{B}_{ji}$ quadrants. The portions of each $\mathbf{A}_{i:m,:}$ and $\mathbf{B}_{:,j:n}$ vector belonging to quadrant $\mathbf{A}_{ij}$ and $\mathbf{B}_{ji}$ are notated as $\mathbf{A}_{ij:i:m,:}$ and $\mathbf{B}_{ji:,j:n}$. The MXU then computes and returns row i of all $\mathbf{C}$ sub-blocks from the lowest level of recursion in (4) in every clock cycle i , allowing $\mathbf{C}_{i:m,:}$ to be stored in the same format as $\mathbf{A}$ in memory for if $\mathbf{C}$ will later be taken as an $\mathbf{A}$ input for a later matrix multiplication.

B. Strassen Multisystolic Array Design

Fig. 2 shows the proposed SMM_r multisystolic array architecture. Rather than having one $X \times Y$ MXU with X columns and Y rows of MAC units for efficiently multiplying matrices down to size $X \times Y$, this architecture consists of 7^r smaller $X/2^r \times Y/2^r$ MXUs that together efficiently multiply matrices down to the same size but at a higher throughput. Furthermore, it achieves this with fewer MAC units than a conventional multisystolic array design. This both allows smaller matrices to be multiplied at a higher utilization and increases the throughput per MAC unit.

The $\mathbf{A}_{i:m,:}$ and $\mathbf{B}_{:,j:n}$ vectors read into the MXU are first divided into their four $\mathbf{A}_{ij:i:m,:}$ and $\mathbf{B}_{ji:,j:n}$ portions

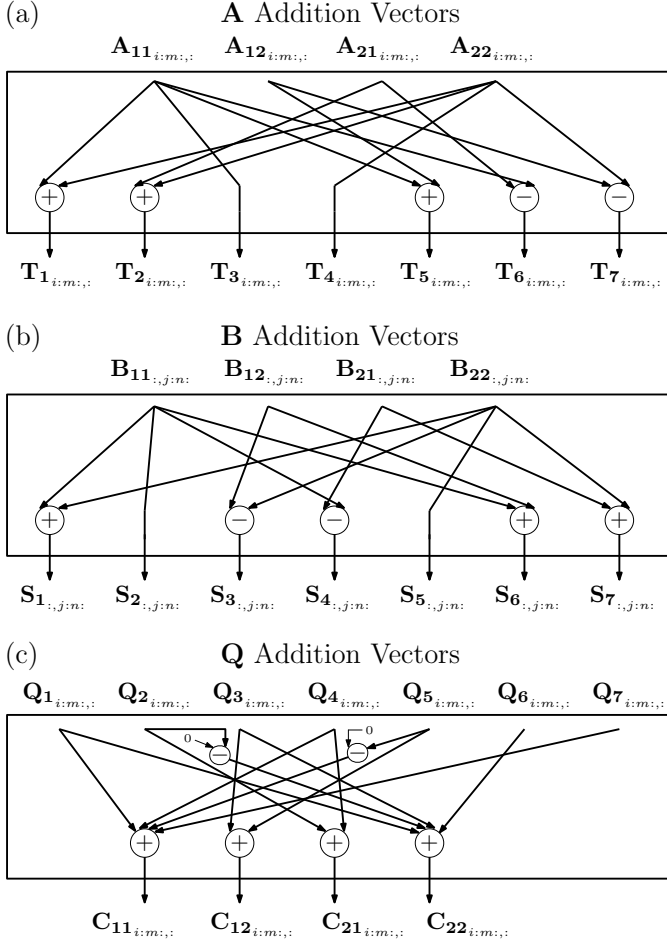


Fig. 3. Internal structure of the SMM_r MXU addition vectors from Fig. 2.

depending on which quadrant of A/B each element belongs to as shown in Fig. 2. They then pass through the A/B addition vectors shown in Fig. 3 (a) and (b) to form the $T_{i:m,:}/S_{:,j:n}$ matrices. The A/B addition vectors both contain 5 addition vectors each consisting of K scalar adders or subtractors, where K is the width of the four A_{ij} blocks and the height of the four B_{ji} blocks as defined in Section III-A. The 7 $T_{i:m,:}/S_{:,j:n}$ vectors then pass into the next level of SMM_{r-1} MXUs to perform the 7 matrix block multiplications. The $Q_{i:m,:}$ vectors of the matrix block multiplication outputs then pass through the $Q_{i:m,:}$ addition vectors shown in Fig. 3 (c) consisting of 8 addition vectors each containing N scalar adders or subtractors. This forms the final C product, where N is the width of the four B_{ji} blocks as defined in Section III-A.

Each of the 7 SMM_{r-1} MXUs can contain 7 more SMM_{r-2} MXUs for implementing another level of Strassen recursion and repeating the process above, or they can be instantiated as a baseline MM MXU shown in Fig. 5. For implementing the next level of SMM_{r-2} MXUs inside each SMM_{r-1} MXU, each $T_{i:m,:}/S_{:,j:n}$ input passed into an SMM_{r-1} MXU will then be considered as the full $A_{i:m,:}/B_{:,j:n}$ inputs within that MXU and are split again into the next level of four $A_{ij:m,:}/B_{ji:,j:n}$ vectors. The dimensions of the matrix blocks

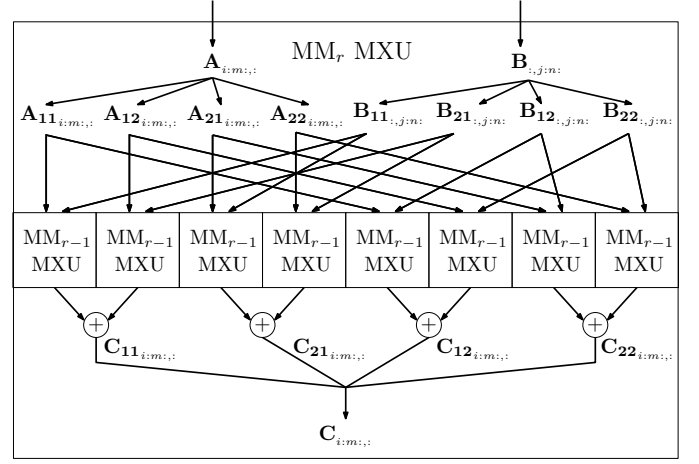


Fig. 4. Baseline MM_r multisystolic array architecture for implementing conventional blocked matrix multiplication (2) for r levels of recursion in hardware.

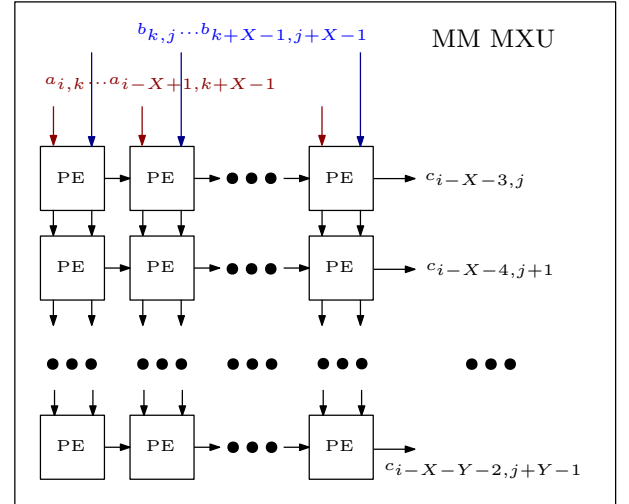


Fig. 5. Baseline MM single-systolic array architecture that implements conventional matrix multiplication (1) in hardware, provided for completeness and clarity. It is instantiated at the lowest level of recursion in the SMM_r and MM_r MXU architectures. X here represents the width of the a and b vectors entering the MM MXU, and Y represents the width of the c vectors exiting the MXU.

being read/computed and the number of scalar adders in the addition vectors within each SMM_{r-1} MXU will then be reduced by a factor of 2 at each level of recursion. For fixed-point implementations, the $T_{i:m,:}/S_{:,j:n}$ inputs to each SMM_{r-1} MXU that were formed from an addition or subtraction in the A or B vector addition units will have an increased bitwidth by 1 bit.

C. Baseline Designs

We later compare the SMM_r architectures with baseline MM_r multisystolic array architectures shown in Fig. 4 which execute (2) in parallel for r levels of recursion. The baseline MM_r architectures are functionally identical to the SMM_r architectures, but they consist of 8^r smaller $X/2^r \times Y/2^r$ MXUs rather than 7^r . Fig. 5 also shows the internal structure

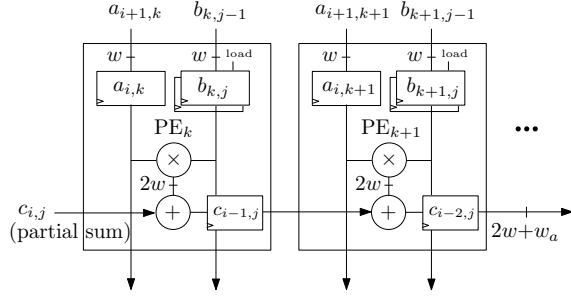


Fig. 6. The internal PE structure of each MM MXU from Fig. 5, provided for completeness. Here, w_a is the additional bitwidth added to account for accumulation, equal to $\lceil \log_2(X) \rceil$, where X is the width of the a and b vectors entering the MM MXU.

of each baseline MM MXU present at the lowest level of recursion in each SMM_r and MM_r architecture, and Fig. 6 shows the internal structure of the processing elements (PE)s inside the MM MXUs.

IV. RESULTS

In this section, we evaluate example implementations of the proposed SMM_r architectures. In Section IV-D, we compare the SMM_r MXU architectures in isolation against our baseline MXU designs in Table I. In Section IV-E we evaluate the SMM_r MXU architectures in Tables II-III compared to prior work when integrated into an end-to-end deep learning accelerator system based on the system from our previous work [6], which has open-source code [23]. We first describe the integration of our proposed systolic arrays into the deep learning system in Section IV-A, and in Sections IV-B and IV-C, we define performance metrics used to compare the SMM_r architectures against baseline designs and prior works.

A. System Integration

We were able to integrate the SMM_r architectures into a system based on our previous work [6], which has open-source code [23], by swapping the SMM_r MXU architectures from Fig. 2 into our system design [6] in place of the free-pipeline fast inner-product (FFIP) MXU.

In order to perform GEMM on the proposed MXUs and multiply matrices of arbitrary sizes that can be larger than the MXU dimensions, the full $\mathbf{A}$ and $\mathbf{B}$ matrices are first divided into GEMM tiles prior to being divided further into smaller blocks for executing (2) or (3)-(4). The GEMM tiles are then fed into the MXU one-by-one. Each GEMM tile is then considered as the full $\mathbf{A}$ and $\mathbf{B}$ matrix from (2) or (3)-(4) while being fed into the MXU and gets further divided into smaller $\mathbf{A}_{ij}/\mathbf{B}_{ji}$ blocks within the MXU.

Following each GEMM tile multiplication, the partial GEMM tile products are accumulated outside of the MXU to generate each final GEMM tile product. Prior to each GEMM tile multiplication, a $\mathbf{B}$ GEMM tile is loaded into the MXU. It then remains in place as the $\mathbf{A}$ GEMM tile flows through the MXU producing the GEMM tile product, during which a new $\mathbf{A}_{i:m,:}$ vector is fed into the MXU each clock cycle. Additionally, to hide the latency of loading $\mathbf{B}$ GEMM tiles,

the MXU PEs each contain one extra b buffer to load the next $\mathbf{B}$ GEMM tile into the MXU as the current GEMM tile is being multiplied.

Each $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ sub-block entering or exiting the top-level MXU for the SMM_r and baseline MXUs first pass through triangular-shaped register arrays each containing X shift registers of varying depths. Here, each shift register SR_k has a depth of k and loads one $a_{i,k}$ or $b_{k,j}$ element per clock cycle. These triangular buffers are explained further in our prior work [6] and they allow the vector elements to enter the MXU in the necessary order as depicted in the element indices in Figs. 5 and 6.

B. Multiplier Compute Efficiency

In this subsection, we define an efficiency metric called the multiplier compute efficiency in (8) which we use to compare the SMM_r architectures against baseline designs and prior works. This is used to quantify how much the algebraic optimizations exploited in an architecture reduce the computational complexity. Reductions in computational complexity allow an architecture to utilize its multipliers more effectively than conventional designs using no algebraic optimizations. The multiplier compute efficiency (MCE) is defined as follows:

$$\text{MCE} = \frac{\text{mults/multiplier}}{\text{clock cycle}} = \frac{(\text{mults/s})/\#\text{multipliers}}{f}. \quad (8)$$

Here, mults/s above is measured by taking the number of multiplications required to carry out an execution using conventional algebra and dividing it by the measured execution time. Finally, $\#\text{multipliers}$ is the number of instantiated multipliers in the design, and f is the clock frequency that the hardware design is operating at.

Conventional matrix multiplication algorithms such as (2) have no algebraic optimizations for reducing the computational complexity. Therefore, the limit/maximum achievable value (also referred to as the roof) of the metric in (8) is the following when using conventional matrix multiplication in hardware:

$$\text{roof}(\text{MCE}_{\text{MM}_r}) = 1. \quad (9)$$

In contrast, Strassen's algorithm requires $8^r/7^r$ times fewer multiplications than a conventional matrix multiplication algorithm, where r is the number of levels of recursion implemented in Strassen's algorithm. Therefore, the multiplier compute efficiency can reach the following limit in SMM_r architectures:

$$\text{roof}(\text{MCE}_{\text{SMM}_r}) = \left(\frac{8}{7}\right)^r. \quad (10)$$

As discussed in Section II, Strassen's algorithm reduces the overall number of operations in matrix multiplication. Furthermore, any additions required before the matrix multiplications in the algorithm are even less of a concern in fixed-point implementations. This is because the hardware complexity of fixed-point multipliers typically scale quadratically with the input bitwidth compared to linearly for adders and registers

[24], [25], [26], causing the hardware footprint of multipliers to dominate that of adders and registers.

However, one of the impediments of using Strassen's algorithm for fixed-point implementations is that the bitwidths of the multiplication inputs increase by r bits for r levels of Strassen recursion that are implemented, reducing its potential area savings for custom fixed-point hardware designs. Nonetheless, this impediment for fixed-point designs can be inherently mitigated in FPGA implementations so long as r plus the initial input width is not larger than the maximum input width supported by the FPGA's DSP units. For example, each DSP in common Intel/Altera FPGAs instantiate two 18×19 -bit multipliers [27], and common input bitwidths for applications such as deep learning are 16 bits or less. This leaves room for at least 2 or more levels of Strassen recursion to be implemented before surpassing the bitwidth limit supported by the DSPs.

Furthermore, due to the flexible nature of custom hardware design, the SMM_r architectures can be efficiently mapped onto other DSP units in general which support input bitwidths up to n bits by customizing the input datapath bitwidth w and value of r as necessary to ensure that $w + r \leq n$. So long as the accuracy requirements of the application are still met, this will allow the SMM_r designs and their increase in multiplier bitwidth to still be efficiently mapped onto DSP units of any bitwidth in a general way.

C. Supporting Smaller Matrices with the Same Performance

Multisystolic array designs such as the SMM_r and baseline MM_r architectures have the ability to efficiently multiply smaller matrices than a single-systolic array design with the same performance capability. By executing (2) or (3)-(4) fully in parallel for r levels of recursion, matrix products of size as small as $n \times n$ can be computed up to once every $n/2^r$ clock cycles in an MM_r or SMM_r multisystolic array design. Furthermore, these matrix products require n^3 multiplications to calculate using conventional algebra. Therefore, the ratio of an architecture's throughput per clock cycle versus its smallest supported matrix sizes it can multiply, which we refer to as the matrix size efficiency (MSE), is the following:

$$\text{MSE} = \frac{\text{mults/clock cycle}}{\text{min. mat. size (h} \times \text{w)}}, \quad (11)$$

which has the following roof for multisystolic arrays:

$$\text{roof}(\text{MSE}_{(S)MM_r}) = \frac{n^3/(n/2^r)}{n \times n} = 2^r. \quad (12)$$

In contrast, a single-systolic array design can produce matrix products of size as small as $n \times n$ up to once every n clock cycles, making this ratio the following for a single-systolic array design:

$$\text{roof}(\text{MSE}_{MM}) = \frac{n^3/n}{n \times n} = 1. \quad (13)$$

This shows that the SMM_r and baseline MM_r multisystolic array designs can efficiently multiply matrices 2^r times smaller than a single-systolic array architecture with the same performance capability.

As discussed in Section II-C, this is an important property for increasing a systolic array accelerator's maximum achievable throughput on real-life workloads. Even if more compute resources are instantiated to scale up the size of the systolic array, the systolic array will begin to be underutilized after its size surpasses the workload's matrix sizes. This is particularly true in modern workloads such as deep learning acceleration, where the matrix sizes that the workloads break down to can be smaller than the maximum systolic array size that could be instantiated in an accelerator [4], [17], [19], [20]. In Section IV-E, we demonstrate how this property allowed us to scale up our deep learning accelerator design without compromising utilization to achieve state-of-the-art ResNet [28] throughput.

D. Comparison to Baseline Designs

Table I shows the resource usage and performance comparison between the proposed SMM_r and baseline MM_r systolic array architectures in isolation (without integration into a deep learning accelerator system). The SMM_1 and SMM_2 architectures overall have a similar amount of soft logic resources and the same throughput per clock cycle roof as the MM_1 and MM_2 architectures, respectively, but they require **1.14-1.31 $\times$ fewer DSP units**. Compared to the multisystolic array MM_1 and MM_2 designs, the SMM_1 and SMM_2 architectures are also functionally equivalent, respectively, other than having a lower clock frequency. To help mitigate the limitation of having a lower frequency, we added an extra SMM_2 design (which had the biggest issue with clock frequency) on the far right of Table I containing additional pipelining registers in the addition logic of each Q Addition Vectors unit from Fig. 3 (c). This extra design demonstrates how a trade-off can be optionally made to increase the design's clock frequency at the cost of some extra soft logic resources.

Nonetheless, the lower clock frequencies of the SMM_r designs in Table I are compensated by the fact that the SMM_r designs achieve more effective operations from the same number DSP units. Since the reduction in DSP units is greater than the reduction in clock frequency in the SMM_1 design and SMM_2 design with extra registers relative to their MM_r counterparts, they would be able to achieve a higher overall throughput if scaled up in size to use the same number of DSPs. This is shown by the Throughput/DSP metric in Table I, which shows that the SMM_r designs achieve up to **22% more throughput per DSP** than their MM_r counterparts. Finally, if the frequency-limiting critical path is in external control or other logic outside of the systolic array after integrating it into an end-to-end accelerator system, as was the case in our full-system accelerators from Tables II-III, this limitation of a lower frequency is further mitigated.

The throughput per clock cycle roof of the MM and MM_2 baseline designs in Table I are equal and they consume the same number of DSP resources, but the MM design requires slightly fewer ALM and register resources. However, this penalty may be justified in the MM_2 design when considering that the minimum matrix size (height $\times$ width) that can be multiplied while fully utilizing the MXU is $4 \times$ smaller in the MM_2 design compared to the MM design. This increases

TABLE I
COMPARING SMM_r MULTISYSTOLIC ARRAY ARCHITECTURES AGAINST THE BASELINE MM SINGLE-SYSTOLIC ARRAY ARCHITECTURE AND BASELINE MM_r MULTISYSTOLIC ARRAY ARCHITECTURES IN ISOLATION (WITHOUT INTEGRATION INTO A DEEP LEARNING ACCELERATOR SYSTEM).

	MM 48×48	MM ₁ 16×16	SMM ₁ 16×16	MM ₂ 6×6	SMM ₂ 6×6	SMM ₂ 6×6 (with extra regs.)
DSPs	1,152	1,024	896	1,152	882	882
ALMs	34,890	30,872	30,265	36,397	35,863	38,485
Registers	130,262	118,049	115,830	138,219	133,511	147,750
Frequency (MHz)	399	398	380	388	291	361
roof (Throughput) (GOPS) ¹	1839	1630	1556	1788	1341	1663
Throughput/DSP ²	1.60	1.59	1.74	1.55	1.52	1.89
roof $\left(\frac{\text{mults/multiplier}}{\text{clock cycle}}\right)$ ³	1	1	1.14	1	1.31	1.31
Min. supported matrix size ⁴	48×48	32×32	32×32	24×24	24×24	24×24
roof $\left(\frac{\text{mults/clock cycle}}{\text{min. mat. size (h×w)}}\right)$ ⁵	1	2	2	4	4	4

All designs are synthesized on Arria 10 GX 1150 FPGA for 16-bit fixed-point inputs and consume 0 memory resources.

¹ Maximum achievable throughput in giga operations per second, where throughput is equal to the number of operations required to carry out an execution using conventional algebra divided by the measured execution time.

² Shows which designs can achieve the highest throughput for the same number of DSPs.

³ Maximum achievable multiplier compute efficiency, defined in Section IV-B, measures how effectively the architecture can utilize its multipliers. It can surpass 1 in SMM_r architectures because the observed mults/s is equal to the number of multiplications required to carry out an execution using conventional algebra divided by execution time.

⁴ Minimum input matrix sizes that can be multiplied at peak throughput/full utilization.

⁵ Quantifies how much smaller the minimum supported matrix sizes of a multisystolic array design are relative to a single-systolic array design with the same throughput per clock cycle roof, definition and relevance provided in Section IV-C.

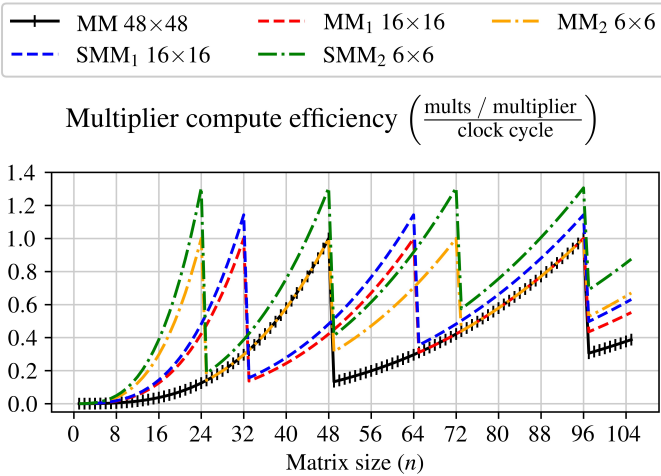


Fig. 7. Plotting the multiplier compute efficiency of the architectures in Table I when multiplying different input matrices of size $n \times n$. As shown, the minimum matrix size that can be efficiently multiplied gets smaller in the order of the MM, MM_1/SMM_1 , and MM_2/SMM_2 designs, respectively.

its performance scalability for accelerating modern workloads such as deep learning as discussed in Section II-C and IV-C. This ability of the multisystolic array designs to more efficiently multiply smaller matrices is further illustrated in Fig. 7. This same property is true for the SMM_2 design, except it achieves this with fewer DSP resources. This benefit is demonstrated in Section IV-E, where this property allowed us to scale up our deep learning accelerator design without compromising utilization to achieve state-of-the-art ResNet throughput.

E. Comparison to Prior Work

Full system-level validation of the experimental accelerator as integrated into the system from our previous work [6] has been done on an Arria 10 SoC Development Kit [29] containing the Arria 10 SX 660 device by measuring throughput in real-time. However, this device contains fewer soft logic resources than the Arria 10 GX 1150 used in the prior works we compare against, and we generate compilation results for our design on the same Arria 10 GX 1150 device used in prior works for a more fair and consistent comparison. Throughput values of our designs on the Arria 10 GX 1150 device are then calculated using an accurate throughput estimation model based on our highly deterministic and time-predictable system implementation, which accurately predicts actual throughputs measured on the Arria 10 SX 660 device available to us. Tables II-III show throughputs for ResNet [28] neural networks.

The works from Liu et al. [30] and Fan et al. [31] in Table II use a technique to pack two 8-bit multiplications onto each 18×19 -bit multiplier in the DSPs and additional ALMs, and therefore the number of multipliers is calculated as $\# \text{DSP} \times 4$ in those works. The number of multipliers in the works [32], [33] from Table IV is calculated as $\# \text{DSPs}$ since they are implemented on AMD FPGAs where each DSP instantiates one 18×27 -bit multiplier [34]. In Tables II and IV, the number of multipliers in the prior works [35], [36] is equal to $\# \text{DSPs} \times 2$, where each DSP in the Altera FPGAs instantiates two 18×19 -bit multipliers [27]. The number of multipliers used in the MXUs from our architectures in Tables I-II is equal to 8^r or 7^r times $X \times Y$ for the MM_r and SMM_r MXUs, respectively. For example, an MM 64×64 MXU (meaning $r = 0$ and $X = Y = 64$) would contain $8^0 \times 64^2$ multipliers, an MM_1 32×32 MXU would contain $8^1 \times 32^2$ multipliers, and an SMM_2 8×8 MXU would contain $7^2 \times 8^2$ multipliers. Due

TABLE II
SMM_r MULTISYSTOLIC ARRAY ARCHITECTURES INTEGRATED INTO A DEEP LEARNING ACCELERATOR SYSTEM COMPARED WITH PRIOR STATE-OF-THE-ART DEEP LEARNING ACCELERATORS.

	TNNLS '22 [30]		TCAD '22 [31]		Entropy '22 [35]		SMM ₁ 32×32			SMM ₂ 8×8		
DSPs	1473		1473		1503		1518			1518		
ALMs	304K		304K		303K		306K			145K		
Registers	889K		890K		-		641K			386K		
Memories	2334		2334		1953		2713			2036		
Frequency (MHz)	200		220		172		293			295		
Model	ResNet-50	VGG-16	Bayes ResNet-18	Bayes VGG-11	R-CNN (ResNet-50)	R-CNN (VGG-16)	ResNet-50	ResNet-101	ResNet-152	ResNet-50	ResNet-101	ResNet-152
Throughput (GOPS) ¹	1519	1295	1590	534	719	865	3750	4116	4276	2024	2115	2158
$\frac{\text{mults/multiplier}}{\text{clock cycle}}$ ²	0.645	0.550	0.639	0.206	0.696	0.837	0.877	0.963	1.002	1.051	1.098	1.120

All designs are synthesized on Arria 10 GX 1150 FPGA for 8-bit fixed-point inputs.

¹ Throughput in giga operations per second, equal to the number of operations required to carry out an execution using conventional algebra divided by execution time.

² Multiplier compute efficiency, defined in Section IV-B, measures how effectively the architecture utilizes its multipliers. It can surpass 1 in SMM_r architectures because the observed mults/s is equal to the number of multiplications required to carry out an execution using conventional algebra divided by the measured execution time. For prior works that did not provide this metric, the value displayed here is reverse engineered based on other provided metrics and design choices as explained further in Section IV-E.

TABLE III
COMPARISON OF AN FFIP [6] SINGLE-SYSTOLIC ARRAY ARCHITECTURE, WHICH DOUBLES PERFORMANCE PER MAC UNIT, WITH COMBINED FFIP+SMM_r MULTISYSTOLIC ARRAY ARCHITECTURES WHEN INTEGRATED INTO DEEP LEARNING ACCELERATOR SYSTEMS.

	TC '24 [6] (FFIP 64×64)			FFIP+SMM ₁ 32×32			FFIP+SMM ₂ 8×8		
DSPs	1072			1518			946		
ALMs	118K			216K			165K		
Registers	311K			627K			463K		
Memories	1782			2713			2036		
Frequency (MHz)	388			313			297		
Model	ResNet-50	ResNet-101	ResNet-152	ResNet-50	ResNet-101	ResNet-152	ResNet-50	ResNet-101	ResNet-152
Throughput (GOPS) ¹	2529	2752	2838	4006	4397	4568	2038	2130	2172
$\frac{\text{mults/multiplier}}{\text{clock cycle}}$ ²	1.521	1.655	1.707	1.674	1.837	1.908	1.813	1.895	1.933

All designs are synthesized on Arria 10 GX 1150 FPGA for 8-bit fixed-point inputs.

¹⁻² See the corresponding definitions from Table II.

to the FFIP reduction in multipliers as described in our prior work [6], the number of multipliers for the FFIP architectures in Table III is equal to 8^r or 7^r times $X \times Y/2 + X/2$ for the FFIP and FFIP+SMM_r designs, respectively. Additionally, for our deep learning accelerator implementations in Tables II-III, there are an additional $Y \times 4^r$ multipliers located outside the MXU in the Post-GEMM Unit [6] for performing inter-layer quantization rescaling functions. For our designs requiring more than 3036 multipliers, 3036 are instantiated on 1518 DSPs, and the remainder are instantiated in soft logic resources as the DSP resources are fully utilized.

Tables II-III show the SMM_r architectures integrated into the deep learning system from our previous work [6] compared to state-of-the-art accelerators evaluated on the same FPGA family for the same input bitwidths and similar neural network models. Integrating the SMM_r multisystolic array design into our deep learning accelerator allowed us to increase the multiplier compute efficiency while also scaling up the computational resources and throughput roof without increasing the minimum supported matrix sizes. This allowed it to significantly surpass the throughput in our prior work

[6] and other state-of-the-art prior works evaluated on the same FPGA family as shown in Tables II-III. If the design is scaled up using a single-systolic array, the minimum supported matrix size increases and compute resources begin to be underutilized for ResNet execution based on the smaller matrix sizes that its workload decomposes to. This causes the effective throughput to not increase well despite the design having a larger throughput roof.

The SMM₁ 32×32 and FFIP+SMM₁ 32×32 designs consume noticeably more memory resources than the SMM₂ 8×8 and FFIP+SMM₂ 8×8 designs. However, it is worth noting that this is not due to increased memory requirements, but rather is due to the compiler favouring to swap some register resources for memory resources. This is because the SMM₁ 32×32 and FFIP+SMM₁ 32×32 designs have a higher register (and overall area) overhead than the SMM₂ 8×8 and FFIP+SMM₂ 8×8 designs in order to achieve higher throughput roofs.

In Table II, the SMM_r architectures achieve the highest throughput and multiplier compute efficiency compared to the prior works. The SMM₁ and SMM₂ architectures' multiplier

TABLE IV

STATE-OF-THE-ART DEEP LEARNING ACCELERATORS IMPLEMENTED ON OTHER FPGA FAMILIES FOR SIMILAR NEURAL NETWORKS AND INPUT BITWIDTHS TO PROVIDE FURTHER COMPARISON WITH TABLES II AND III.

	TNNLS '22 [32]	TCAS-I '23 [33]	TCAD '24 [36]
FPGA	AMD VX980	AMD XCV U9P	Altera Stratix 10 GX650
Fixed-point input bitwidth	8/16 ³	8	8
DSPs	3121	2048	1024
ALMs (Altera) / LUTs (AMD)	480K	-	152K
Registers	-	-	567K
Memories (20Kb Altera) / (36Kb AMD)	1457	-	2056
Frequency (MHz)	100	200	200
Model	ResNet-101	ResNet-50	ResNet-152
Throughput (GOPS) ¹	600	287	794
$\frac{\text{mults/multiplier}}{\text{clock cycle}}$ ²	0.961	0.351	0.969

¹⁻² See the corresponding definitions from Table II.

³ Weights are quantized to 8 bits and layer input/output is quantized to 8 or 16 bits at different stages.

compute efficiencies in Table II approach their limits of 1.14 and 1.31 that are derived in (10). This surpasses the limit of 1 of the baseline MM_r architectures and prior works that is derived in (9), validating SMM_r 's ability to increase multiplier compute efficiency and reduce computational complexity as expected from our analysis.

Table III shows an example of how SMM_r can be combined with other algebraic techniques to further increase multiplier compute efficiency limits. FFIP [6] provides a way to reduce the number of required multiplications by up to a factor of 2, trading half the multiplications for cheap low-bitwidth additions. Because of this, the limit for the multiplier compute efficiency metric in (8) for an FFIP architecture becomes 2, and $2 \times (8/7)^r$ for a combined FFIP+ SMM_r architecture. In Table III, we evaluate architectures that combine FFIP+ SMM_r by instantiating SMM_r MXUs that use FFIP MXUs at their lowest level of recursion instead of the conventional MM MXUs from Fig. 5. This further increases multiplier compute efficiency compared to a standalone SMM_r or standalone FFIP MXU as seen in the achieved multiplier compute efficiencies of the FFIP+ SMM_r architectures listed in Table III.

V. CONCLUSION

Strassen's fast matrix multiplication algorithm reduces the complexity of naive matrix multiplication, however, general-purpose hardware is not suitable for achieving the algorithm's promised theoretical speedups. Furthermore, there is limited prior work on custom hardware architectures designed specifically for executing the algorithm in hardware. We address this by presenting custom Strassen multisystolic array hardware architectures that are functionally equivalent to conventional multisystolic array designs. However, they allow the theoretical complexity reductions of Strassen's algorithm to be

translated directly into hardware resource savings, even for multiplication of small matrices.

Compared to a conventional multisystolic array design, the proposed architectures implemented on FPGA for 1 and 2 levels of Strassen recursion use $1.14\times$ and $1.31\times$ fewer DSP units and an overall comparable amount of soft logic resources when instantiated for multiplying $n \times n$ matrices down to sizes $n = 32$ and $n = 24$, respectively. The proposed systolic array architectures increase conventional multiplications/multiplier/clock cycle limits by a factor of 1.14^r for r implemented levels of Strassen recursion. Furthermore, they allow the throughput per clock cycle roof of an accelerator to double for each implemented level of Strassen recursion without increasing the minimum supported matrix sizes that can be efficiently multiplied.

REFERENCES

- [1] A. Fuchs and D. Wentzlaff, "The accelerator wall: Limits of chip specialization," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2019, pp. 1–14.
- [2] X. Liu *et al.*, "WinoCNN: Kernel sharing Winograd systolic array for efficient convolutional neural network acceleration on FPGAs," in *Proc. IEEE 32nd Int. Conf. Appl.-Specific Syst., Arch. Processors (ASAP)*, 2021, pp. 258–265.
- [3] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2017, pp. 1–12.
- [4] T. Norrie *et al.*, "The design process for Google's training chips: TPUv2 and TPUv3," *IEEE Micro*, vol. 41, no. 2, pp. 56–63, 2021.
- [5] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, 2016, pp. 4013–4021.
- [6] T. E. Pogue and N. Nicolici, "Fast inner-product algorithms and architectures for deep neural network accelerators," *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 495–509, 2024.
- [7] V. Strassen, "Gaussian elimination is not optimal," *Numer. Math.*, vol. 13, no. 4, pp. 354–356, 1969.
- [8] J. Sun *et al.*, "Accelerate dense matrix multiplication on heterogeneous-GPUs," in *2023 IEEE 29th Int. Conf. Parallel and Distrib. Syst. (ICPADS)*, 2023, pp. 2726–2729.
- [9] C. Misra *et al.*, "Stark: Fast and scalable Strassen's matrix multiplication using apache spark," *IEEE Trans. Big Data*, vol. 8, no. 3, pp. 699–710, 2022.
- [10] A. G. Krishnan and D. Goswami, "Multi-stage memory efficient Strassen's matrix multiplication on GPU," in *2021 IEEE 28th Int. Conf. on High Perform. Comput., Data, and Anal. (HiPC)*, 2021, pp. 212–221.
- [11] A. Khaled *et al.*, "Applying fast matrix multiplication to neural networks," in *Proc. 35th Annu. ACM Symp. on Appl. Comput.*, 2020, pp. 1034–1037.
- [12] J. Huang *et al.*, "Strassen's algorithm reloaded on GPUs," *ACM Trans. on Math. Softw. (TOMS)*, vol. 46, no. 1, pp. 1–22, 2020.
- [13] P.-W. Lai *et al.*, "Accelerating Strassen-Winograd's matrix multiplication algorithm on GPUs," in *IEEE 20th Annu. Int. Conf. on High Perf. Comput.*, 2013, pp. 139–148.
- [14] B. Lipshitz *et al.*, "Communication-avoiding parallel Strassen: Implementation and performance," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2012, pp. 1–11.
- [15] G. Ballard *et al.*, "Communication-optimal parallel algorithm for Strassen's matrix multiplication," in *Proc. 24th Annu. ACM Symp. Parallelism Algorithms Archit.*, 2012, pp. 193–204.
- [16] S. Winograd, "On multiplication of 2×2 matrices," *Linear Algebra Appl.*, vol. 4, no. 4, pp. 381–388, 1971.
- [17] N. Jouppi *et al.*, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proc. 50th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2023, pp. 1–14.
- [18] C. Zhang *et al.*, "Caffeine: Toward uniformed representation and acceleration for deep convolutional neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 11, pp. 2072–2085, 2019.

- [19] H. T. Kung *et al.*, “Maestro: A memory-on-logic architecture for coordinated parallel use of many systolic arrays,” in *Proc. IEEE 32nd Int. Conf. Appl.-Specific Syst., Arch. Processors (ASAP)*, vol. 2160-052X, 2019, pp. 42–50.
- [20] A. C. Yüzügüler *et al.*, “Scale-out systolic arrays,” *ACM Trans. Archit. Code Optim.*, vol. 20, no. 2, mar 2023.
- [21] L. Elfimova and Y. V. Kapitonova, “A fast algorithm for matrix multiplication and its efficient realization on systolic arrays,” *Cybern. Syst. Anal.*, vol. 37, no. 1, pp. 109–121, Jan. 2001.
- [22] L. G. León-Vega *et al.*, “Acceleration of fully connected layers on FPGA using the Strassen matrix multiplication,” in *Proc. IEEE 5th Int. Conf. BioInspired Processing (BIP)*, 2023, pp. 1–6.
- [23] T. E. Pogue and N. Nicolici, “FFIP accelerator implementation,” 2024. [Online]. Available: <https://github.com/trevorpogue/algebraic-nnhw>
- [24] V. Lakshmi *et al.*, “A novel in-memory wallace tree multiplier architecture using majority logic,” *IEEE Trans. Circuits Syst. I*, vol. 69, no. 3, pp. 1148–1158, 2022.
- [25] K. Guo *et al.*, “[DL] a survey of FPGA-based neural network inference accelerators,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, pp. 1–26, 2019.
- [26] K. Pekmezci, “Multiplexer-based array multipliers,” *IEEE Trans. Comput.*, vol. 48, no. 1, pp. 15–23, 1999.
- [27] “Intel Arria 10 native fixed point DSP IP core user guide,” 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/programmable/683583/current/intel-arria-native-fixed-point-dsp-ip.html>
- [28] K. He *et al.*, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [29] “Intel Arria 10 SoC development kit,” 2024.
- [30] S. Liu *et al.*, “Toward full-stack acceleration of deep convolutional neural networks on FPGAs,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 8, pp. 3974–3987, 2022.
- [31] H. Fan *et al.*, “FPGA-based acceleration for bayesian convolutional neural networks,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 12, pp. 5343–5356, 2022.
- [32] W. Huang *et al.*, “FPGA-based high-throughput CNN hardware accelerator with high computing resource utilization ratio,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 8, pp. 4069–4083, 2022.
- [33] D. Kim *et al.*, “Agamotto: A performance optimization framework for CNN accelerator with row stationary dataflow,” *IEEE Trans. Circuits Syst. I*, vol. 70, no. 6, pp. 2487–2496, 2023.
- [34] “UltraScale architecture DSP slice,” 2024. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug579-ultrascale-dsp>
- [35] J. An *et al.*, “An OpenCL-based FPGA accelerator for Faster R-CNN,” *Entropy*, vol. 24, no. 10, p. 1346, 2022.
- [36] K. Dai *et al.*, “DCP-CNN: Efficient acceleration of CNNs with dynamic computing parallelism on FPGA,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2024.



Nicola Nicolici (S’99-M’00-SM’11) Nicola Nicolici (S99-M00-SM’11) received the Dipl.Ing. degree in Computer Engineering from the “Politehnica” University of Timisoara, Romania, in 1997 and the Ph.D. degree in Electronics and Computer Science from the University of Southampton, U.K., in 2000. He is currently a Professor with the Department of Electrical and Computer Engineering, McMaster University, Hamilton, Canada. His research interests are in the area of computer-aided design and test. He has authored a number of papers in this area. Dr.

Nicolici was the recipient of the IEEE TTTC Beausang Award for the Best Student Paper at the International Test Conference in 2000 and the Best Paper Award at the IEEE/ACM Design Automation and Test in Europe Conference in 2004.



Trevor E. Pogue Trevor E. Pogue received the B.Eng. degree in Electrical Engineering and the M.A.Sc. degree in Electrical and Computer Engineering from McMaster University, Hamilton, Canada, in 2016 and 2019, respectively. He worked as an intern at Synopsys and AMD in 2018 and 2022-2023, respectively. He is currently a Ph.D. Candidate in the Department of Electrical and Computer Engineering at McMaster University, Hamilton, Canada. His research interests are in the area of hardware acceleration.